



DEPARTAMENTO DE ENGENHARIA
ELÉTRICA – DEE

CENTRO DE ENGENHARIA ELÉTRICA E INFORMÁTICA – CEEI



Desenvolvimento de um Processador RISC-V baseado em uma Arquitetura de Pipeline Assíncrono com Dados Agrupados

Vinícius de França Nobrega

Dr. Gutemberg Gonçalves dos Santos Júnior

23096.005521/2025-01

12 de Agosto de 2026



Sumário

Introdução

Contextualização

Objetivos

Fundamentação Teórica

RISC-V

Pipelines Assíncronos com Dados

Agrupados

Metodologia

Concepção (*Design*)

Ambiente de Simulação

Síntese Lógica

Implementação Física

Resultados Obtidos

Considerações Finais

Contextualização

- ▶ Demanda por dispositivos de alta **eficiência energética** e baixo consumo (*low-power*)



Figura 1: Internet das Coisas (*IoT*, Fonte: *igzy.com*).

Contextualização

- ▶ Demanda por dispositivos de alta **eficiência energética** e baixo consumo (*low-power*)
- ▶ Pipelines síncronos dependem de clock global -> *clock tree* impacta consumo

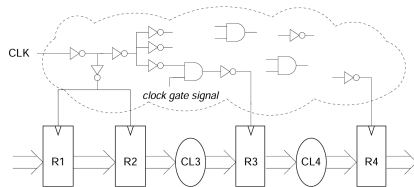


Figura 2: Ilustração de distribuição de clock síncrono.

Contextualização

- ▶ Demanda por dispositivos de alta **eficiência energética** e baixo consumo (*low-power*)
- ▶ Pipelines síncronos dependem de clock global -> *clock tree* impacta consumo
- ▶ Arquiteturas **assíncronas** surgem como alternativa (SUTHERLAND, 1989).

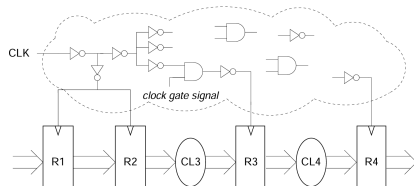


Figura 2: Ilustração de distribuição de clock síncrono.

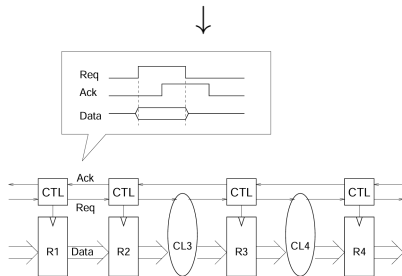


Figura 3: Ilustração de pipeline assíncrono de dados agrupados (SPARSo; FURBER, 2001).

Contextualização

- ▶ Demanda por dispositivos de alta **eficiência energética** e baixo consumo (*low-power*)
- ▶ Pipelines síncronos dependem de clock global -> *clock tree* impacta consumo
- ▶ Arquiteturas **assíncronas** surgem como alternativa (SUTHERLAND, 1989).
- ▶ RISC-V permite arquiteturas customizadas



Figura 4: RISC-V (riscv.org).



Objetivos

Projetar e implementar um processador RISC-V baseado em uma arquitetura de **pipeline assíncrono com dados agrupados** (*bundled-data*), avaliando seu **desempenho, área e consumo** de potência em **comparação** com uma implementação síncrona equivalente.

Objetivos

Projetar e implementar um processador RISC-V baseado em uma arquitetura de **pipeline assíncrono com dados agrupados** (*bundled-data*), avaliando seu **desempenho, área e consumo** de potência em **comparação** com uma implementação síncrona equivalente.

Objetivos Específicos

- ▶ Implementar versão assíncrona básica

Objetivos

Projetar e implementar um processador RISC-V baseado em uma arquitetura de **pipeline assíncrono com dados agrupados** (*bundled-data*), avaliando seu **desempenho, área e consumo** de potência em **comparação** com uma implementação síncrona equivalente.

Objetivos Específicos

- ▶ Implementar versão assíncrona básica
- ▶ Explorar otimizações para versão assíncrona

Objetivos

Projetar e implementar um processador RISC-V baseado em uma arquitetura de **pipeline assíncrono com dados agrupados** (*bundled-data*), avaliando seu **desempenho, área e consumo** de potência em **comparação** com uma implementação síncrona equivalente.

Objetivos Específicos

- ▶ Implementar versão assíncrona básica
- ▶ Explorar otimizações para versão assíncrona
- ▶ Implementar versões síncronas de referência

Objetivos

Projetar e implementar um processador RISC-V baseado em uma arquitetura de **pipeline assíncrono com dados agrupados** (*bundled-data*), avaliando seu **desempenho, área e consumo** de potência em **comparação** com uma implementação síncrona equivalente.

Objetivos Específicos

- ▶ Implementar versão assíncrona básica
- ▶ Explorar otimizações para versão assíncrona
- ▶ Implementar versões síncronas de referência
- ▶ Submeter todas as versões ao fluxo de ASIC (síntese, PnR, etc.)

Objetivos

Projetar e implementar um processador RISC-V baseado em uma arquitetura de **pipeline assíncrono com dados agrupados** (*bundled-data*), avaliando seu **desempenho, área e consumo** de potência em **comparação** com uma implementação síncrona equivalente.

Objetivos Específicos

- ▶ Implementar versão assíncrona básica
- ▶ Explorar otimizações para versão assíncrona
- ▶ Implementar versões síncronas de referência
- ▶ Submeter todas as versões ao fluxo de ASIC (síntese, PnR, etc.)
- ▶ Avaliar e comparar abordagens: desempenho vs. custo (área, consumo)

Contextualização Teórica - RISC-V

- ▶ ISA aberta modular
- ▶ RV32I
- ▶ Pipeline de 5 estágios: *Fetch, Decode, Execute, Memory, Write Back* (HARRIS; HARRIS, 2022)



Formato	Bits	[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]	Uso típico
R	funct7	rs2	rs1	funct3	rd	opcode		Operações reg-reg
I	imm[11:0]		rs1	funct3	rd	opcode		Loads, imediatos
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode		Stores
B	imm[12,10:5]	rs2	rs1	funct3	imm[4:1,11]	opcode		Salto condicionais
U		imm[31:12]				rd	opcode	LUI, AUIPC
J		imm[20,10:1,11,19:12]				rd	opcode	JAL

Figura 5: Subconjunto de instruções RISC-V - RV32I (WATERMAN; ASANOVIĆ, 2019).

Pipeline Tradicional - 5 estágios

Pipeline de 5 estágios
(*Fetch, Decode, Execute, Memory, Write Back*).

Podemos separá-lo em:

- ▶ Caminho de Dados (*Datapath*): circuito que processa instruções.
- ▶ Caminho de Controle (*Controlpath*): circuito que gera sinais de *clock* para o *datapath*.

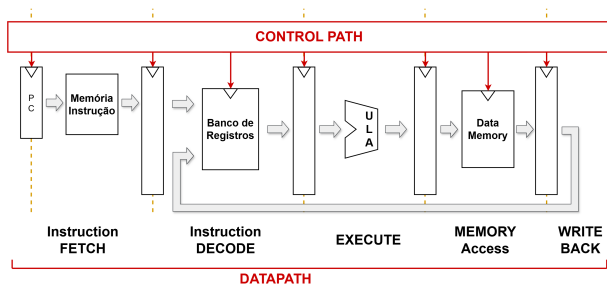


Figura 6: Pipeline de 5 estágios tradicional.

Pipelines Assíncronos com Dados Agrupados

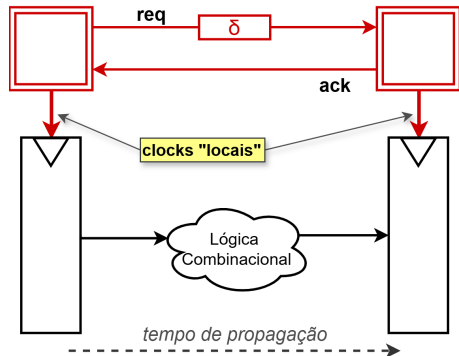


Figura 7: Estágio de Pipeline de Dados Agrupados
(*Bundled-Data*).

- ▶ Proposto por:
(SUTHERLAND, 1989).
- ▶ Clock global substituído por controladores que geram **clock locais**.
- ▶ Comunicação por *handshake*
- ▶ Cada estágio opera na velocidade dos dados

Fluxo de Desenvolvimento ASIC

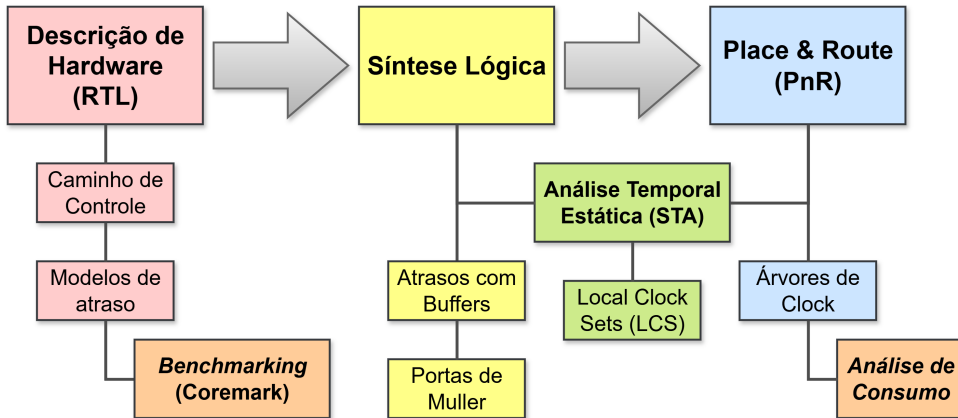


Figura 8: Fluxo de Desenvolvimento ASIC abordado neste projeto.

Arquiteturas Implementadas - Síncrona Básica

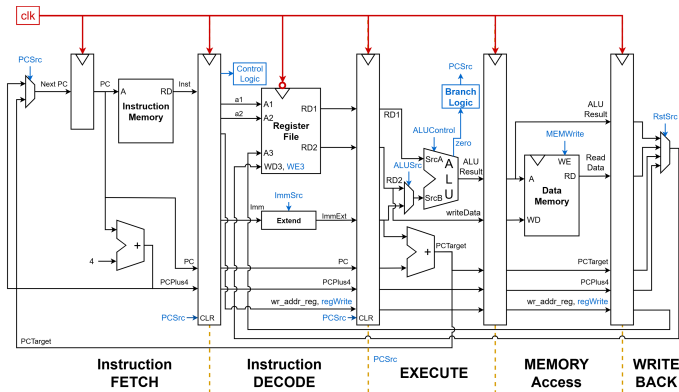


Figura 9: Arquitetura Síncrona Básica.

- ▶ Pipeline básico para RV32I.
- ▶ **Nenhum tratamento** de *hazards*: depende de bolhas (NOP) inseridas pelo compilador.

Arquiteturas Implementadas - Assíncrona Básica

- ▶ Nenhum tratamento de *hazards*: 1 instrução processada por vez entre estágios dependentes.
- ▶ Caminho de Controle com **protocolo simples** (WCHB).

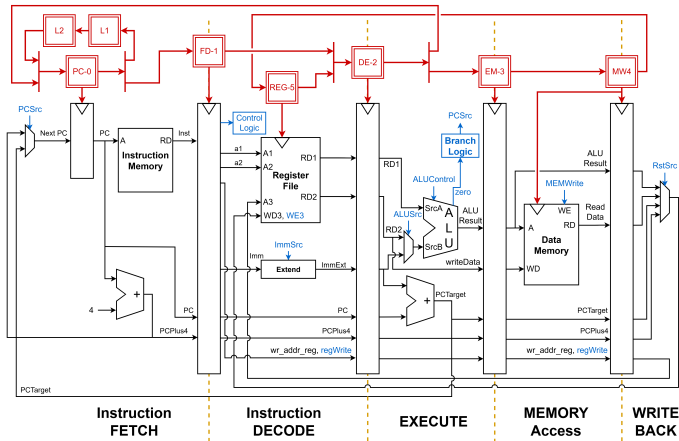
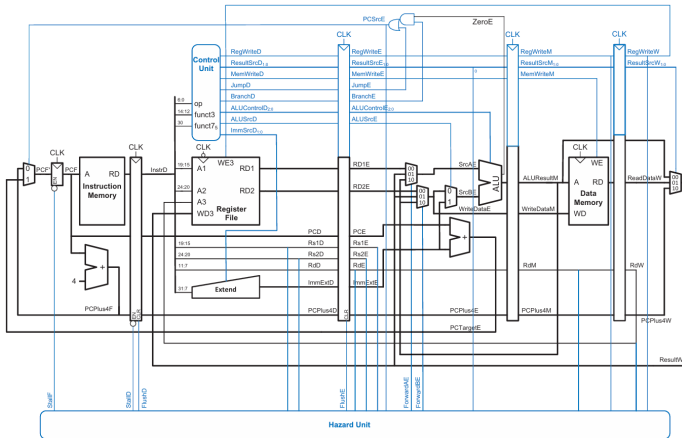


Figura 10: Arquitetura Assíncrona Básica.

Arquiteturas Implementadas - Síncrona Otimizada



- ▶ Introdução da Unidade de Tratamento de *Hazards*.
- ▶ Microarquitetura completa proposta por Harris (HARRIS; HARRIS, 2022).

Figura 11: Arquitetura Síncrona Otimizada (HARRIS; HARRIS, 2022).

Arquiteturas Implementadas - Assíncrona Otimizada

- ▶ Introdução da Unidade de **Tratamento de Hazards**, permitindo processamento de mais instruções em estágios dependentes.
- ▶ Caminho de Controle com **protocolo complexo: Late-Capture** (SIMATIC et al., 2016).

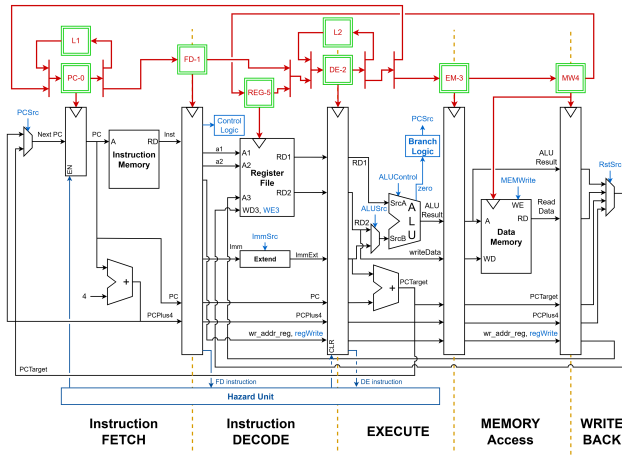


Figura 12: Arquitetura Assíncrona Otimizada.



Ambiente de Simulação

A verificação funcional do processador foi conduzida por **simulação RTL** utilizando *Verilator*.



Ambiente de Simulação

A verificação funcional do processador foi conduzida por **simulação RTL** utilizando *Verilator*.

Três tipos de **programas de teste** foram utilizados:

Ambiente de Simulação

A verificação funcional do processador foi conduzida por **simulação RTL** utilizando *Verilator*.

Três tipos de **programas de teste** foram utilizados:

- ▶ **Sequência de Instruções:**
 - ▶ cobertura manual de toda ISA
 - ▶ auto grau de depuração

Ambiente de Simulação

A verificação funcional do processador foi conduzida por **simulação RTL** utilizando *Verilator*.

Três tipos de **programas de teste** foram utilizados:

- ▶ **Sequência de Instruções:**

- ▶ cobertura manual de toda ISA
- ▶ auto grau de depuração

- ▶ **Programa Assembly básico:**

- ▶ cobertura manual de toda ISA
- ▶ capaz de depurar compilador (*RISC-V GNU Compiler Toolchain*)

Ambiente de Simulação

A verificação funcional do processador foi conduzida por **simulação RTL** utilizando *Verilator*.

Três tipos de **programas de teste** foram utilizados:

- ▶ **Sequência de Instruções:**
 - ▶ cobertura manual de toda ISA
 - ▶ auto grau de depuração
- ▶ **Programa Assembly básico:**
 - ▶ cobertura manual de toda ISA
 - ▶ capaz de depurar compilador (*RISC-V GNU Compiler Toolchain*)
- ▶ **Programa de Benchmark:**
 - ▶ *Coremark*: programa complexo para *benchmarking* (EEMBC, 2009)
 - ▶ auto grau de validação funcional

Síntese Lógica e Implementação Física

- Síntese Lógica realizada com PDK genérico 45nm Cadence.

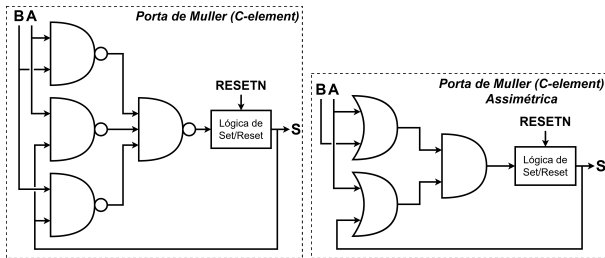


Figura 13: Circuitos para Portas de Muller.

Síntese Lógica e Implementação Física

- ▶ Síntese Lógica realizada com PDK genérico 45nm *Cadence*.
- ▶ STA realizada segundo método LCS (GIMENEZ et al., 2018).

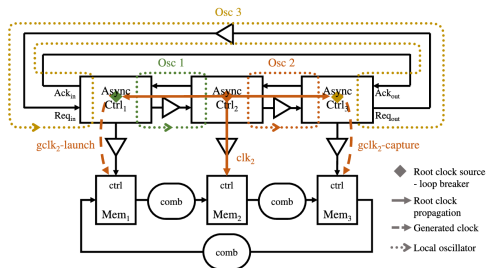


Figura 14: Ilustração do Método *Local Clock Sets* (GIMENEZ et al., 2018).

Síntese Lógica e Implementação Física

- ▶ Síntese Lógica realizada com PDK genérico 45nm *Cadence*.
- ▶ STA realizada segundo método LCS (GIMENEZ et al., 2018).
- ▶ Implementação Física (PnR) básica realizada em *Innovus*.

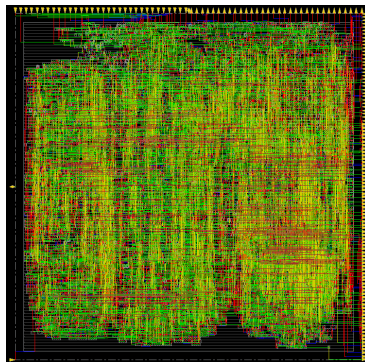


Figura 15: Layout do processador implementado (assíncrono otimizado) em *Innovus*.

Síntese Lógica e Implementação Física

- ▶ Síntese Lógica realizada com PDK genérico 45nm *Cadence*.
- ▶ STA realizada segundo método LCS (GIMENEZ et al., 2018).
- ▶ Implementação Física (PnR) básica realizada em *Innovus*.
- ▶ Circuito final submetido à simulação e análise de consumo com *Joules*.

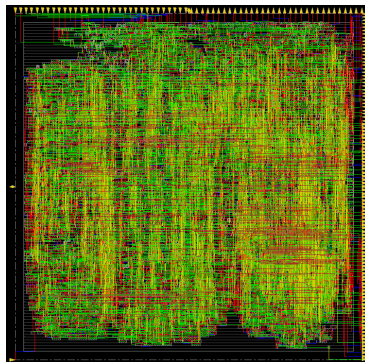


Figura 15: Layout do processador implementado (assíncrono otimizado) em *Innovus*.

Resultados de Desempenho

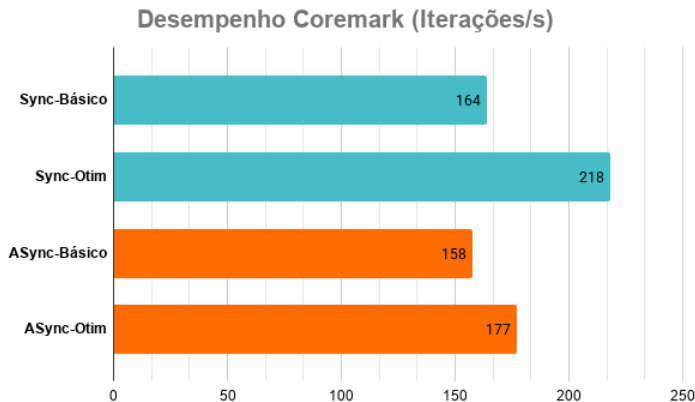


Figura 16: Resultados de Desempenho (Coremark).

- ▶ Versão otimizada da arquitetura assíncrona recupera parte da falta de desempenho.
- ▶ Versão **Síncrona** é **mais rápida** ($\approx 20\%$).

Resultados de Área Ocupada

- Implementação assíncrona manteve **área semelhante** à versão síncrona equivalente.

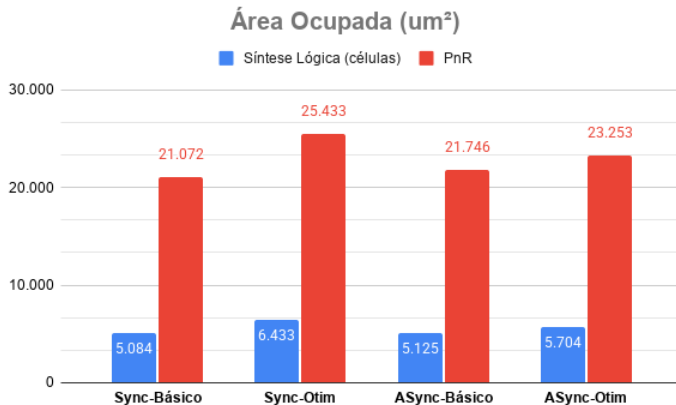


Figura 17: Gráfico de Área Ocupada (Síntese e PnR).

Análise de Consumo

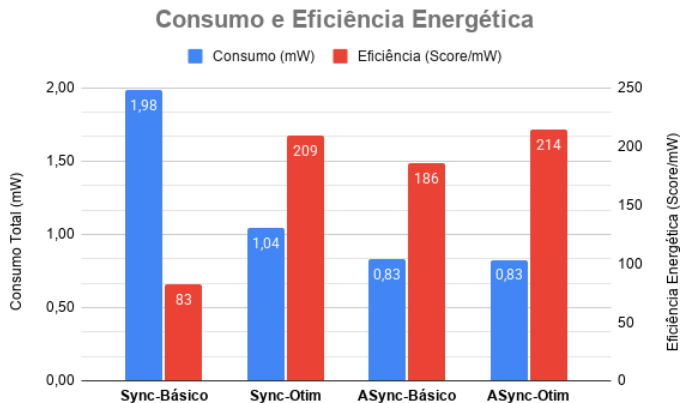


Figura 18: Gráfico de Consumo e Eficiência Energética.

- ▶ $\approx 20\%$ menor consumo de potência.
- ▶ Melhor eficiência energética.

Síntese de Resultados

Pipeline Assíncrono Síntese de Resultados	
Funciona?	✓
Fluxo ASIC?	✓
Melhor Desempenho?	✗
Área Ocupada	✓
Eficiência Energética	✓

Síntese de Resultados

Pipeline Assíncrono Síntese de Resultados	
Funciona?	✓
Fluxo ASIC?	✓
Melhor Desempenho?	✗
Área Ocupada	✓
Eficiência Energética	✓

O pipeline assíncrono mostrou-se uma **alternativa viável** para aplicações em que **eficiência energética** é mais importante que desempenho absoluto.



Perspectivas

Potenciais melhorias:

- ▶ **Multiplexação** do caminho de controle
- ▶ *Forwarding* assíncrono
- ▶ Fabricação em tecnologia real



Perspectivas

Potenciais melhorias:

- ▶ **Multiplexação** do caminho de controle
- ▶ *Forwarding* assíncrono
- ▶ Fabricação em tecnologia real

Contrapontos:

- ▶ **Complexidade** de concepção e manutenção.



Obrigado!

Vinícius de França Nobrega
Dr. Gutemberg Gonçalves dos Santos Júnior
12 de Agosto de 2026

Referências

- EEMBC. *CoreMark*®. 2009. Embedded Microprocessor Benchmark Consortium. Disponível em: <<https://www.eembc.org/coremark/>>.
- GIMENEZ, G. et al. Static timing analysis of asynchronous bundled-data circuits. In: *Proceedings of the 24th IEEE International Symposium on Asynchronous Circuits and Systems (ASYNC)*. [S.l.]: IEEE, 2018. p. 110–118.
- HARRIS, S. L.; HARRIS, D. *Digital Design and Computer Architecture: RISC-V Edition*. [S.l.]: Morgan Kaufmann, 2022. ISBN 978-0-12-820064-3.
- WATERMAN, A.; ASANOVIĆ, K. (Ed.). *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*. Document version 20191214-draft. [S.l.], 2019.
- SIMATIC, J. et al. New asynchronous protocols for enhancing area and throughput in bundled-data pipelines. In: *Proceedings of the 29th Symposium on Integrated Circuits and Systems Design (SBCCI)*. [S.l.]: IEEE, 2016. p. 1–6.
- SPARSO, J.; FURBER, S. *Principles of Asynchronous Circuit Design: A Systems Perspective*. Boston: Kluwer Academic Publishers, 2001. ISBN 978-0-7923-7613-3.
- SUTHERLAND, I. E. Micropipelines. *Communications of the ACM*, v. 32, n. 6, p. 720–738, jun. 1989.